

Project synthesizer Concepten

Analoge & digitale inputs



Pulse soundworks

Docenten:

- Marco Winkelman
- Richard Rosing

Plaats en datum:

- Windesheim T4
- 25 september 2024

Project leden:

- | | |
|---------------------|----------|
| - Rick Smelt | S1198107 |
| - Patrick van Ommen | S1196990 |



w

Herhaling

Achtergronden

wat is de opdracht?

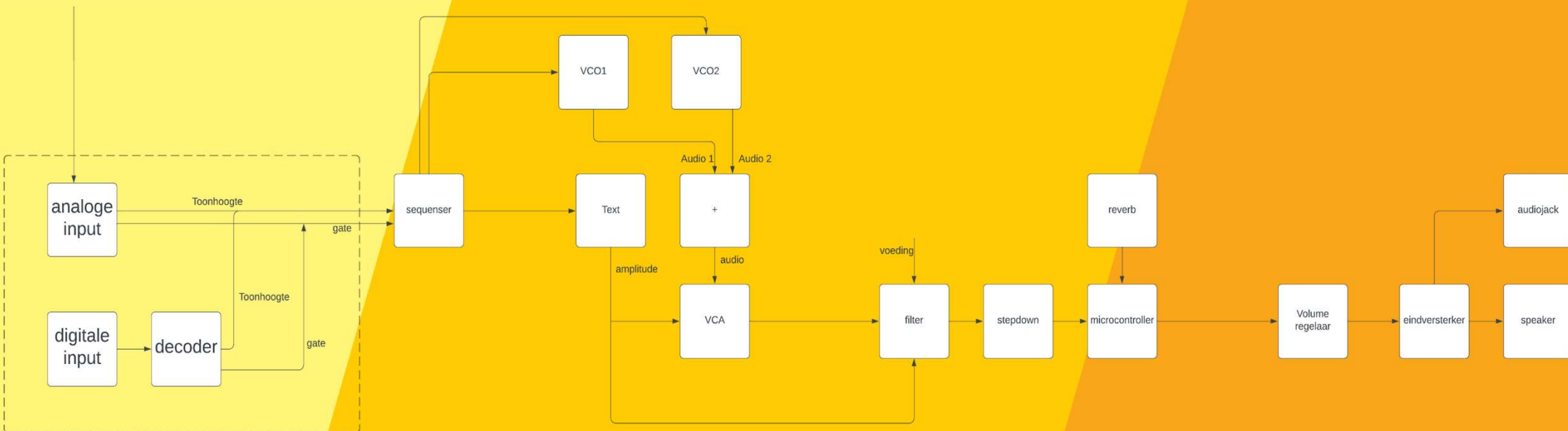
- ☐ Het ontwerpen en bouwen van een modulaire synthesizer.
- ☐ Er moet een analoge input en een midi input worden gemaakt en deze moet aangesloten worden.
- ☐ Voor de analoge input wordt een keyboard gemaakt. En voor de midi input zal het digitale signaal worden omgezet naar een analoog signaal.
- ☐ Uit beide inputs zal een gate signaal komen en een toonhoogte. Deze beide signalen moeten naar de volgende modules:
 - ADSR- module
 - Sequenser
 - oscillator



Project resultaten

Doelstelling:

❑ Aan het eind van dit project is het de bedoeling dat er doormiddel van een midi connector een keyboard kan worden aangesloten. Ook moet er aan het eind van het project een werkende analoge ingang zijn, beide inputs moeten een analoog signaal uitsturen naar de oscillator, de sequenser en de ADSR module.



Analoge input

Eisen Analoge input

Functioneel of niet functioneel	eis
Functioneel	Al is een toets ingedrukt en er word een volgende toets ingedrukt dient deze toon te worden afgespeeld en de initiële toon te stoppen.
Functioneel	Minimaal 1 octaaf aan toetsen waarbij er 4 octaven kunnen worden afgespeeld
Functioneel	Een analoog signaal voor de toonhoogte.
Functioneel	Een gate signaal moet worden verstuurd zodra een toets op het keyboard is ingedrukt of is losgelaten, dit signaal is uit: 0V en aan 5V
Functioneel	1 octaaf krijgt een range van 1 volt.
Niet functioneel	Geen aanslag gevoeligheid
Niet functioneel	Een maximaal bedrag van 50 euro
Niet functioneel	De module moet samen kunnen werken met de andere groepen.
Niet functioneel	Het moet veilig en gemakkelijk in gebruik zijn



w

Digitale input

Eisen Digitale input

Functioneel of niet functioneel	eis
Functioneel	Al is een toets ingedrukt en er word een volgende toets ingedrukt dient deze toon te worden afgespeeld en de initiële toon te stoppen.
Functioneel	De decoder zet het digitale midi signaal om naar een analoog signaal voor de toonhoogte.
Functioneel	Een gate signaal moet worden verstuurd zodra een toets op het keyboard is ingedrukt of is losgelaten, dit signaal is uit: 0V en aan: tussen de 5 en 10V
Functioneel	Na de midi connector moet het digitale signaal worden verstuurt naar een decoder deze zet het midi signaal om naar een analoge signaal die overeen komt met het analoge input signaal
Functioneel	De midi input moet voor het grootste gedeelte voldoen aan het NEN-EN-INC 63035 protocol, daaronder valt dus geen aanslag gevoeligheid en opslaan van het signaal.
Niet functioneel	Geen aanslag gevoeligheid
Niet functioneel	Een maximaal bedrag van 50 euro
Niet functioneel	De module moet samen kunnen werken met de andere groepen.
Niet functioneel	Het moet veilig en gemakkelijk in gebruik zijn



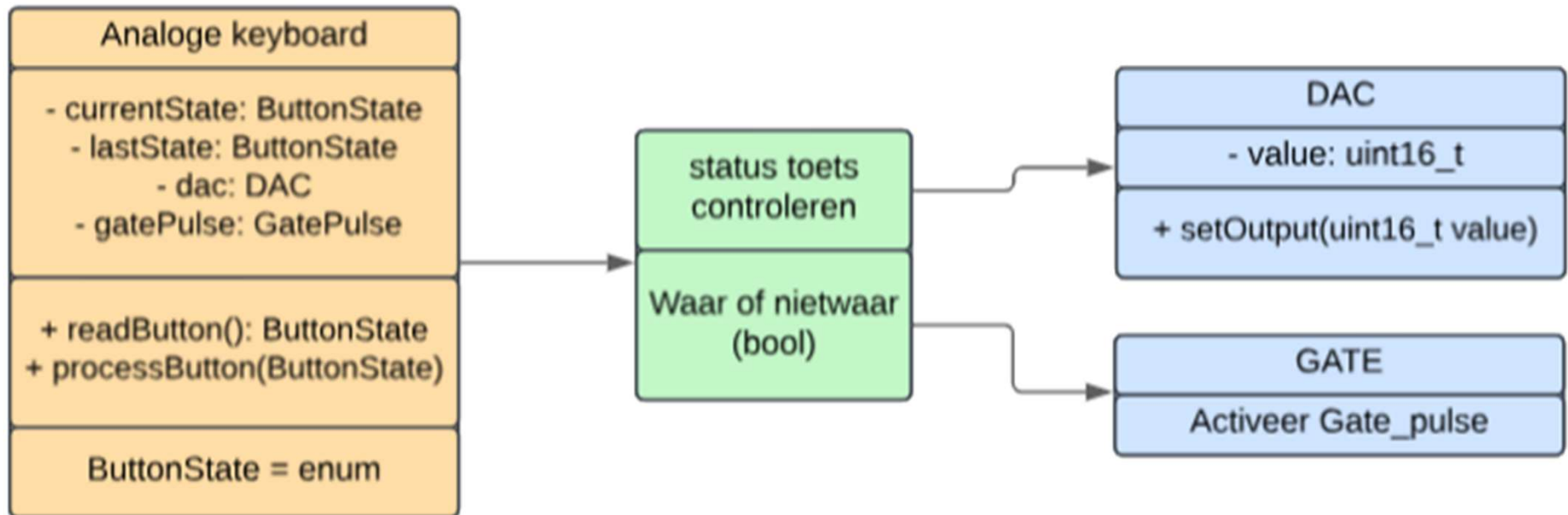
w

Software

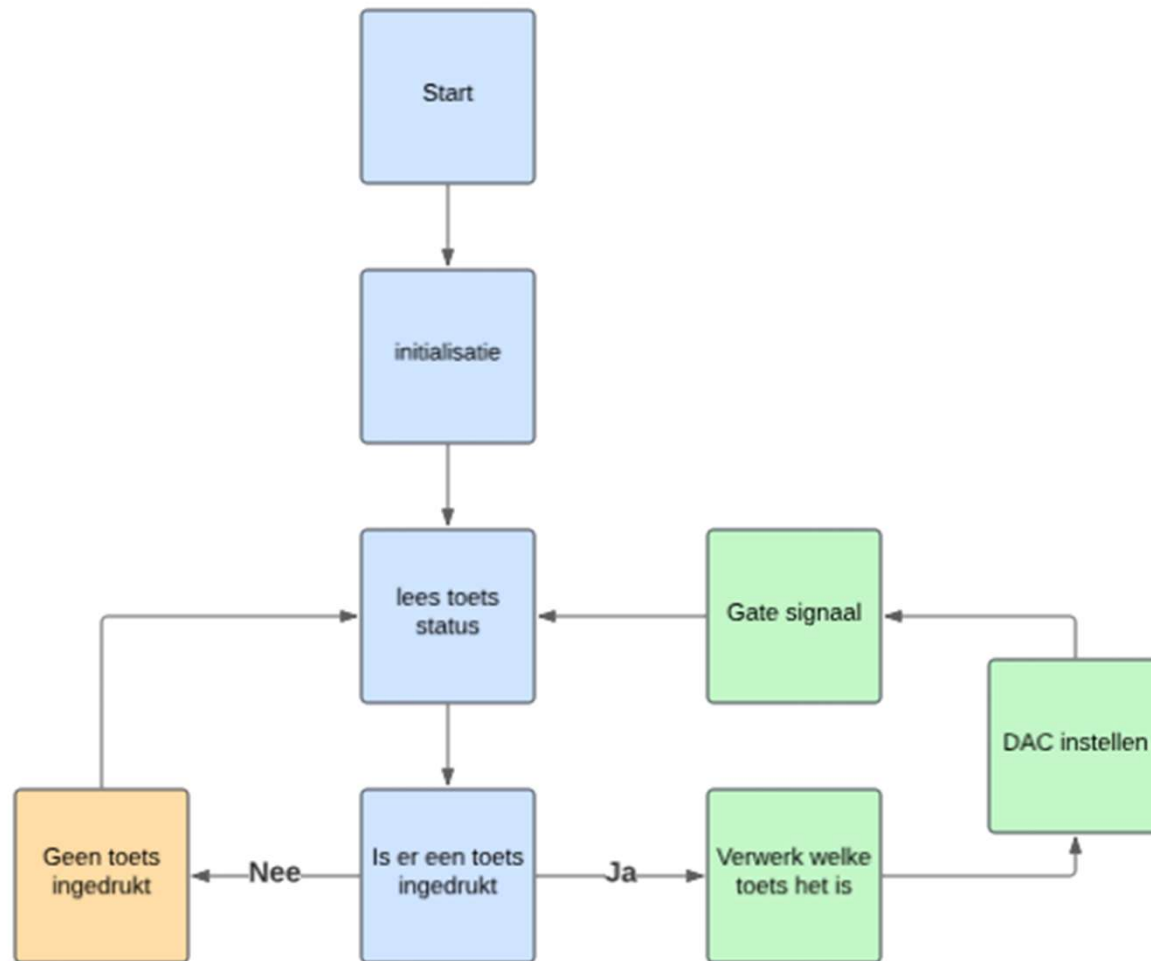
**Eerst Analooog
daarna Digitaal**



Class diagram



Flowchart



Analoge input Code

Gebruikte libraries

```
1 #include "mcc_generated_files/system/system.h"
2 #include <util/delay.h>
```

Functie voor het Gate signaal

```
12 void gate_pulse(void) {
13     gate_SetLow();
14     _delay_ms(10);
15     gate_SetHigh();
16 }
```

Instellen van de DAC

```
6 uint16_t a = 0;
7
8 void DAC_SetOutput(uint16_t value) {
9     DAC0.DATA = value;
10 }
```

States aanmaken

```
18 typedef enum {
19     IDLE,
20     BUTTON_1,
21     BUTTON_2,
22     BUTTON_3,
23     BUTTON_n
24 } ButtonState;
```



Analoge input Code

(PART 2)

Gebruik van IF- statements

```
48 int main(void)
49 {
50     SYSTEM_Initialize();
51     DAC_SetOutput(0);
52     ButtonState currentState = IDLE;
53     ButtonState lastState = IDLE;
54
55     while (1)
56     {
57         if (OCT3_C_GetValue()) {
58             currentState = BUTTON_25;
59         }
60         else if (OCT2_B_GetValue()) {
61             currentState = BUTTON_24;
62         }
63         else if (toets_GetValue()) {
64             currentState = BUTTON_n;
65         }
66         else {
67             currentState = IDLE;
68         }
69     }
70 }
```

Gebruik van switch-case

```
if (currentState != lastState)
{
    switch (currentState)
    {
        case BUTTON_25:
            a = 937;
            DAC_SetOutput(a << 6);
            gate_pulse();
            break;

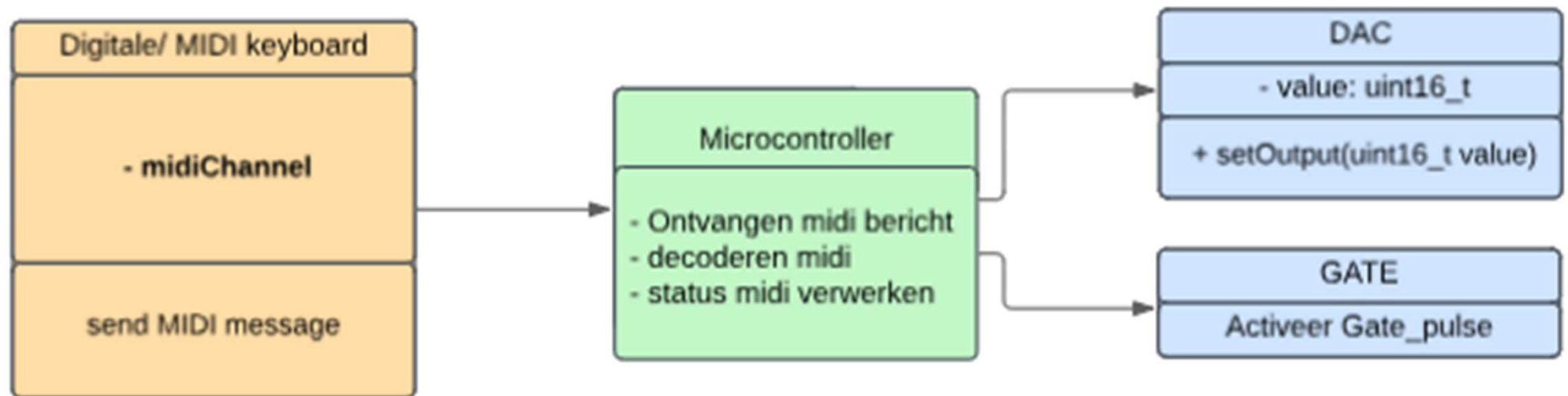
        case BUTTON_25:
            a = 0 - 1023;
            DAC_SetOutput(a << 6);
            gate_pulse();
            break;

        case IDLE:
        default:
            a = 0;
            DAC_SetOutput(a << 6);
            gate_SetLow();
            break;
    }

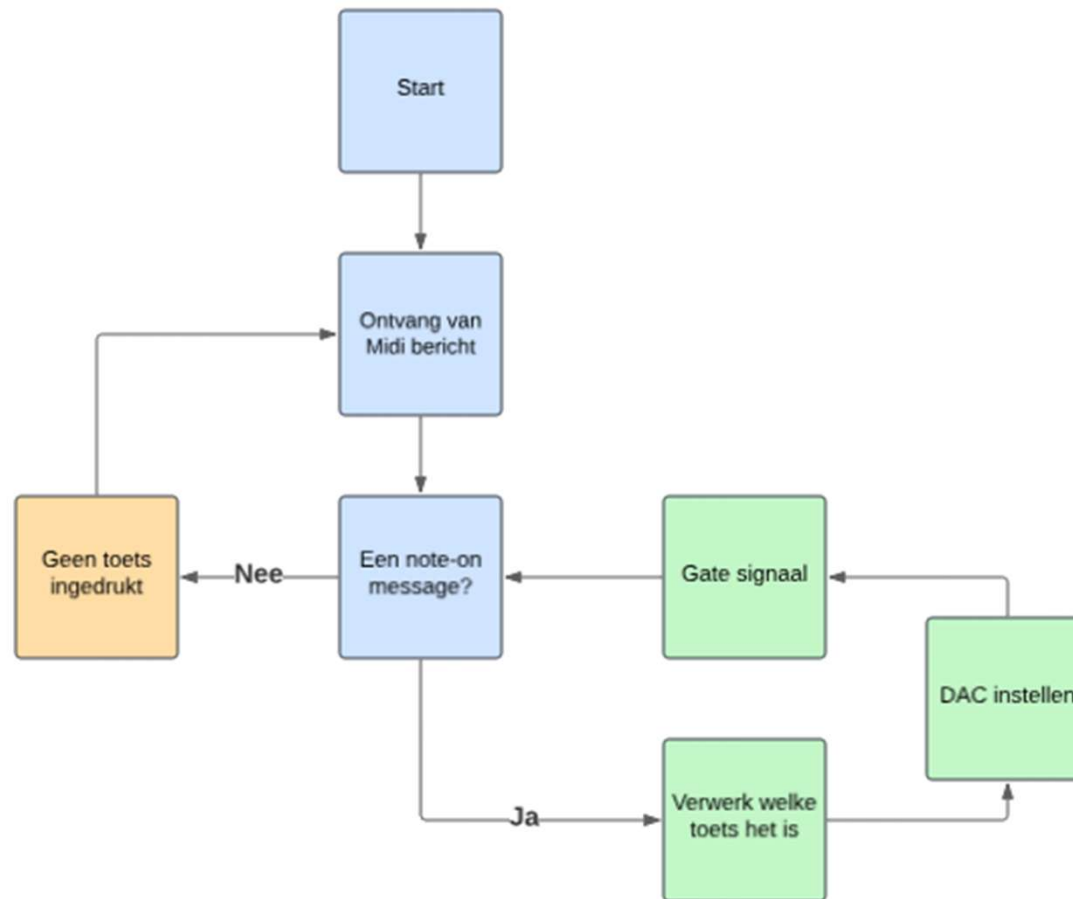
    lastState = currentState;
}
```



Class diagram



Flowchart



Digitale input Code

Gebruikte libraries

```
#include <xc.h> // Inclusie van hardware-specifieke definities voor AVR128DB48.
#include <stdint.h> // Voor standaard integer types zoals uint8_t.
#include <stdbool.h> // Voor boolean typen.
#include "mcc_generated_files/system/system.h" // MCC gegenereerd systeeminitiatiedefinities.
#include "mcc_generated_files/uart/uart0.h" // MCC gegenereerde UART0-ondersteuningsbibliotheek.
#include <util/delay.h> // Voor het gebruik van '_delay_ms()' functies.
```

Functie voor het midi

```
49
50 void USART0_RXC_ISR(void) {
51     USART0_RX_ISR(); // Roep je eigen MIDI-verwerkingsfunctie aan
52 }
```

Hier wordt het midi signaal omgezet

```
void processMIDIMessages(uint8_t status, uint8_t data1, uint8_t data2) {
    // Verwerk NOTE ON berichten
    if ((status & 0xF0) == MIDI_NOTE_ON && data1 > 0 && data2 >= MIDI_MIN_NOTE && data2 <= MIDI_MAX_NOTE) {
        keyStates[data1 - MIDI_MIN_NOTE] = 1; // Zet de corresponderende toetsstatus op ingedrukt.
    }
    // Verwerk NOTE OFF berichten
    else if ((status & 0xF0) == MIDI_NOTE_OFF && data1 > 0 && data2 >= MIDI_MIN_NOTE && data2 <= MIDI_MAX_NOTE) {
        keyStates[data1 - MIDI_MIN_NOTE] = 0; // Zet de corresponderende toetsstatus op losgelaten.
    }
}
```

Hier wordt de uart0 receivepin uitgelezen

Main functie waar wordt geschakelt tussen midi en analoge input

```
DAC_SetOutput(0); // Zet de DAC-uitgang op 0 bij de start.
USART0_SetRxInterruptHandler(USART0_RX_ISR); // Stel de UART RX-interrupt handler in.

while (1) { // Onschuldige lus.
    uint8_t state = Schakelaar_GetValue(); // Lees de waarde van de modus-schakelaar.

    for (uint8_t i = 0; i < 25; i++) { // Itereer over de 25 toetsen.
        if (state && keyStates[i]) { // Als in MIDI-modus en de toets is ingedrukt.
            a = 937 - i * 27; // Bereken de DAC-uitgang voor de toets.
            DAC_SetOutput(a << 6); // Zet de DAC-uitgang.
            gate_pulse(); // Activeer de gate-puls.
            break; // Verlaat de lus zodra een toets is verwerkt.
        } else if (!state) { // Als in analoge modus.
```

```
74
75 void USART0_RXC_ISR(void) {
76     static uint8_t midiStatus = 0, midiData1 = 0; // Variabelen om de status en eerste databyte op te slaan.
77     static bool waitingForData2 = false; // Hulpvariabele om te volgen welke byte verwacht wordt.
78
79     if (USART0_IsRxReady()) { // Controleer of er gegevens beschikbaar zijn in de UART-buffer.
80         uint8_t byte = USART0_Read(); // Lees de inkomende byte.
81
82         if (byte & 0x80) { // Controleer of het een statusbyte is (MSB = 1).
83             midiStatus = byte; // Sla de statusbyte op.
84             waitingForData2 = false; // Reset de datastatus.
85         } else if (!waitingForData2) { // Als het de eerste databyte is.
86             midiData1 = byte; // Sla de eerste databyte op.
87             waitingForData2 = true; // Wacht op de tweede databyte.
88         } else {
89             processMIDIMessages(midiStatus, midiData1, byte); // Verwerk het complete MIDI-bericht.
90             waitingForData2 = false; // Reset voor het volgende bericht.
91         }
92     }
93 }
```



Wat zijn de vragen?

Hopelijk niet te moeilijk